

- 人工智能实验报告 第11周
 - 一.实验题目：在CartPole环境中实现DQN算法
 - 二.实验内容
 - 1.算法原理
 - 2.关键代码展示(可选)
 - 三.实验结果及分析
 - 1.实验结果展示示例
 - 2.评测指标展示及分析
 - 四.参考资料：

人工智能实验报告 第11周

姓名:陈安康 学号:22320021

一.实验题目：在CartPole环境中实现DQN算法

二.实验内容

1.算法原理

深度Q网络（DQN）算法是深度强化学习中的一种经典算法,在高维状态空间上实现了较好的性能表现，尤其在处理复杂的任务如Atari游戏上表现出色。DQN主要思想是将 $Q(s,a)$ 作为在当前环境下选取更好的动作的依据。选取当前状态下 $Q(s,a)$ 取值最大的action作为选择的动作。DQN利用深度神经网络来逼近Q函数，将高维状态输入映射到每个动作的Q值。DQN通过训练神经网络来更新Q值，从而找到最优策略。在这种情况下，当state与action确定时，Q值由神经网络的参数唯一确定，所以将Q值写为： $Q(s,a,\theta)$ ，更新Q值，其实就是更新参数 θ 的过程。

θ 采用TD差分进行更新，更新公式如下：

$$\theta_{t+1} = \theta_t + \alpha[r + \gamma \max_{a'} Q(s', a', \theta) - Q(s, a; \theta)] \nabla Q(s, a; \theta)$$

其中 γ 为折扣因子， α 为学习率， $\nabla Q(s,a;\theta)$ 为Q值对 θ 的梯度。

在实际编写过程中，采用采用 ϵ -贪心策略选择动作，即以 ϵ 的概率随机选择动作，以 $1-\epsilon$ 的概率选择Q值最大的动作。

同时为了使样本满足假设独立，并且提高样本效率，采用经验回放的方法，创建一个回放缓冲区，存储历史数据，等到训练 Q 网络的时候再从回放缓冲区中随机采样若干数据来进行训练。

一般情况下，我们有一个用于选择动作的Q网络（称为主Q网络），同时也有一个目标Q网络。目标Q网络的参数是主Q网络的一种延迟更新版本，它的参数不会像主Q网络那样频繁更新。通过定期更新目标Q网络的参数，可以降低Q值估计的波动性，从而提升训练的稳定性和收敛性。

2.关键代码展示(可选)

训练代码展示如下，主要完成对参数 θ 的更新。

```
def train(self):
    """
    Implement your training algorithm here
    实现训练算法
    """
    #####
    # YOUR CODE HERE #
    #####
    if self.replay_buffer.__len__() < self.mini_size:
        return

    batch = self.replay_buffer.sample(self.batch_size)
    states, actions, rewards, next_states, dones = zip(*batch)

    states = torch.tensor(states, dtype=torch.float32).to(self.device)
    actions = torch.tensor(actions, dtype=torch.int64).unsqueeze(1).to(self.device)
    rewards = torch.tensor(rewards, dtype=torch.float32).unsqueeze(1).to(self.device)
    next_states = torch.tensor(next_states, dtype=torch.float32).to(self.device)
    dones = torch.tensor(dones, dtype=torch.float32).unsqueeze(1).to(self.device)

    q_values = self.q_network(states).gather(1, actions)
    # next_q_values = self.target_q_network(next_states).max(1, keepdim=True)[0]
    next_q_values = self.target_q_network(next_states).max(1)[0].view(-1, 1)
    target_q_values = rewards + (self.gamma * next_q_values * (1 - dones))

    loss = self.loss_fn(q_values, target_q_values.detach())
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()

    if self.step % self.target_update == 0:
        self.target_q_network.load_state_dict(self.q_network.state_dict())

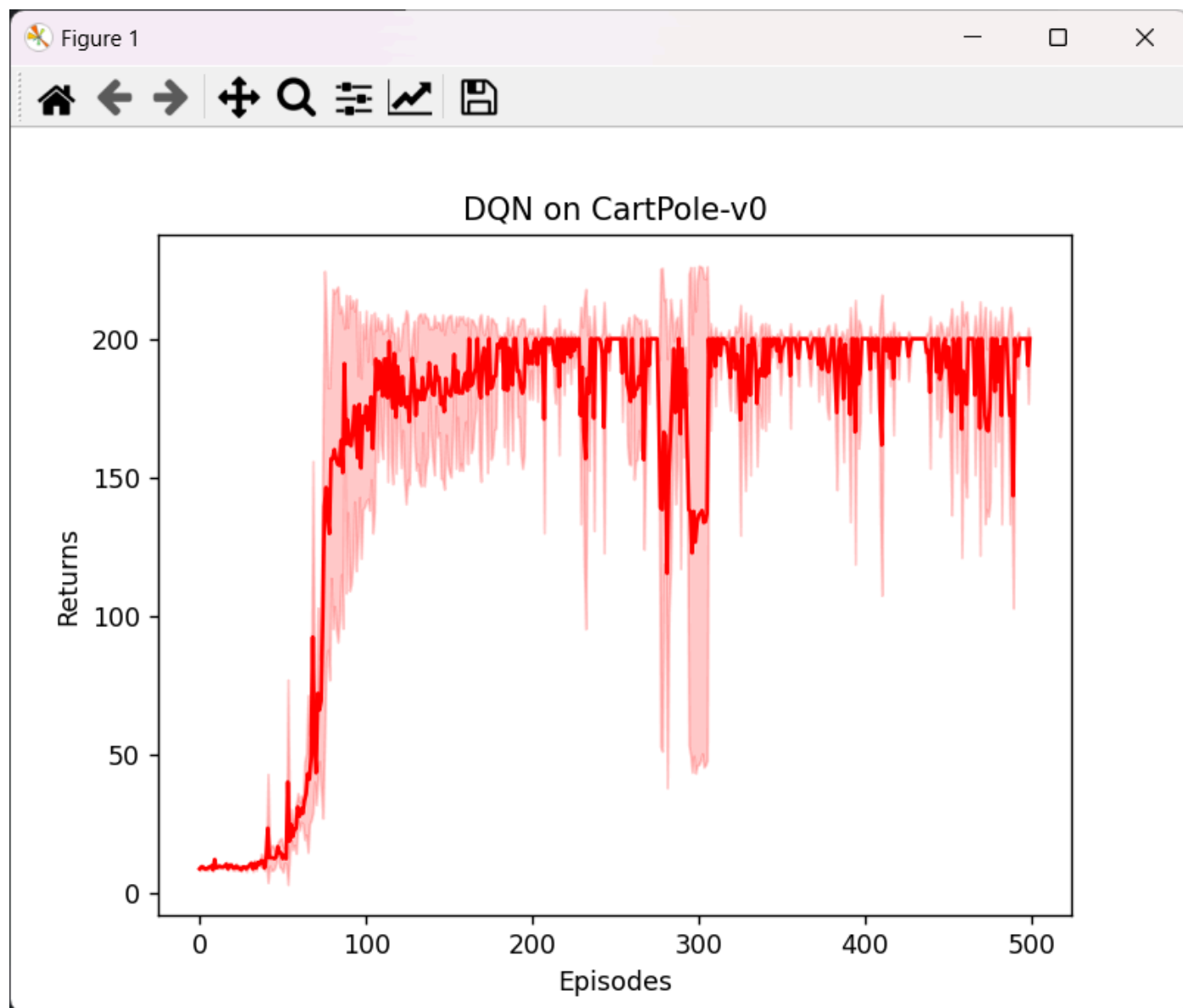
    self.step += 1
```

三.实验结果及分析

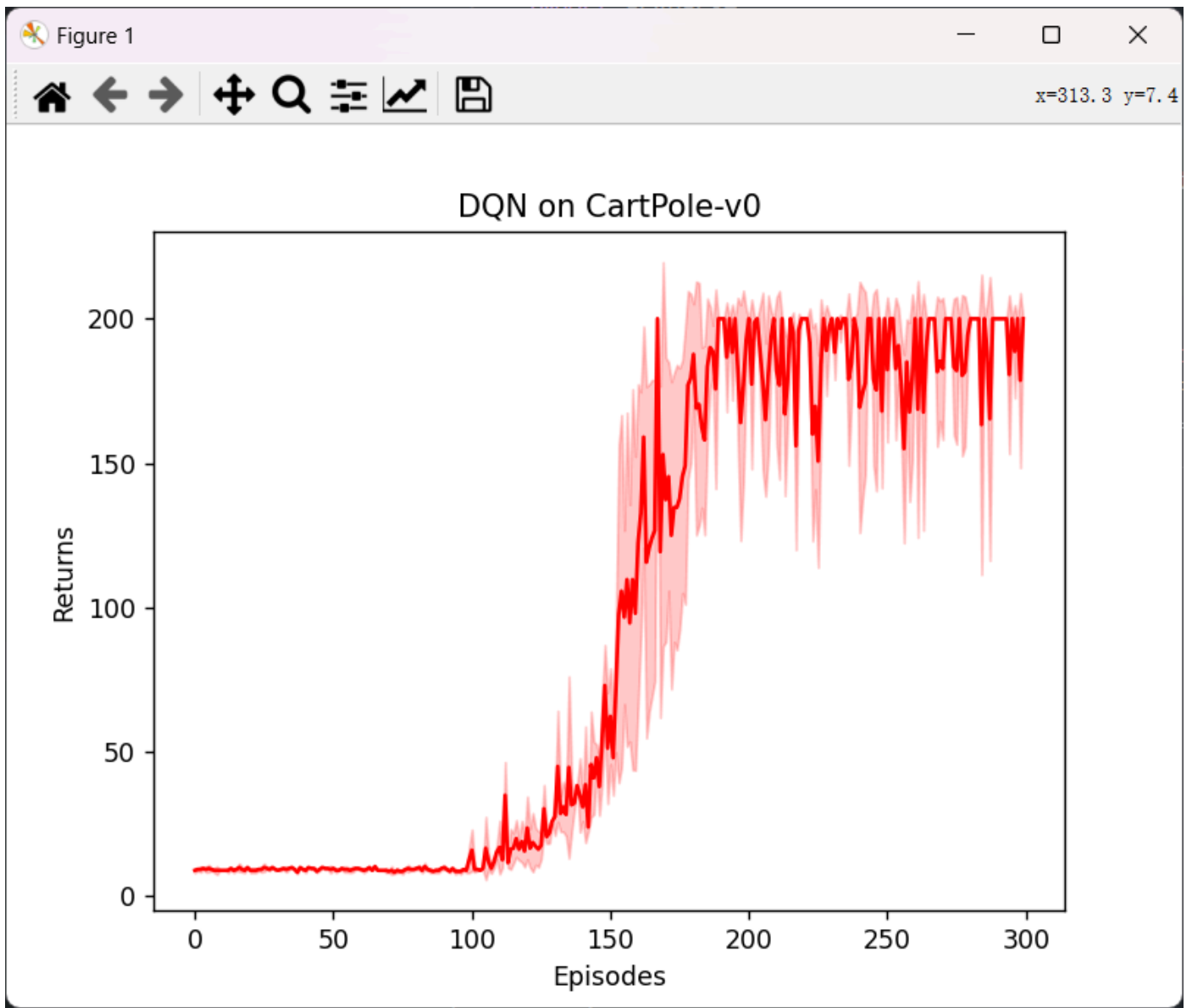
1.实验结果展示示例

迭代500次，使用样本为64的情况下进行训练。

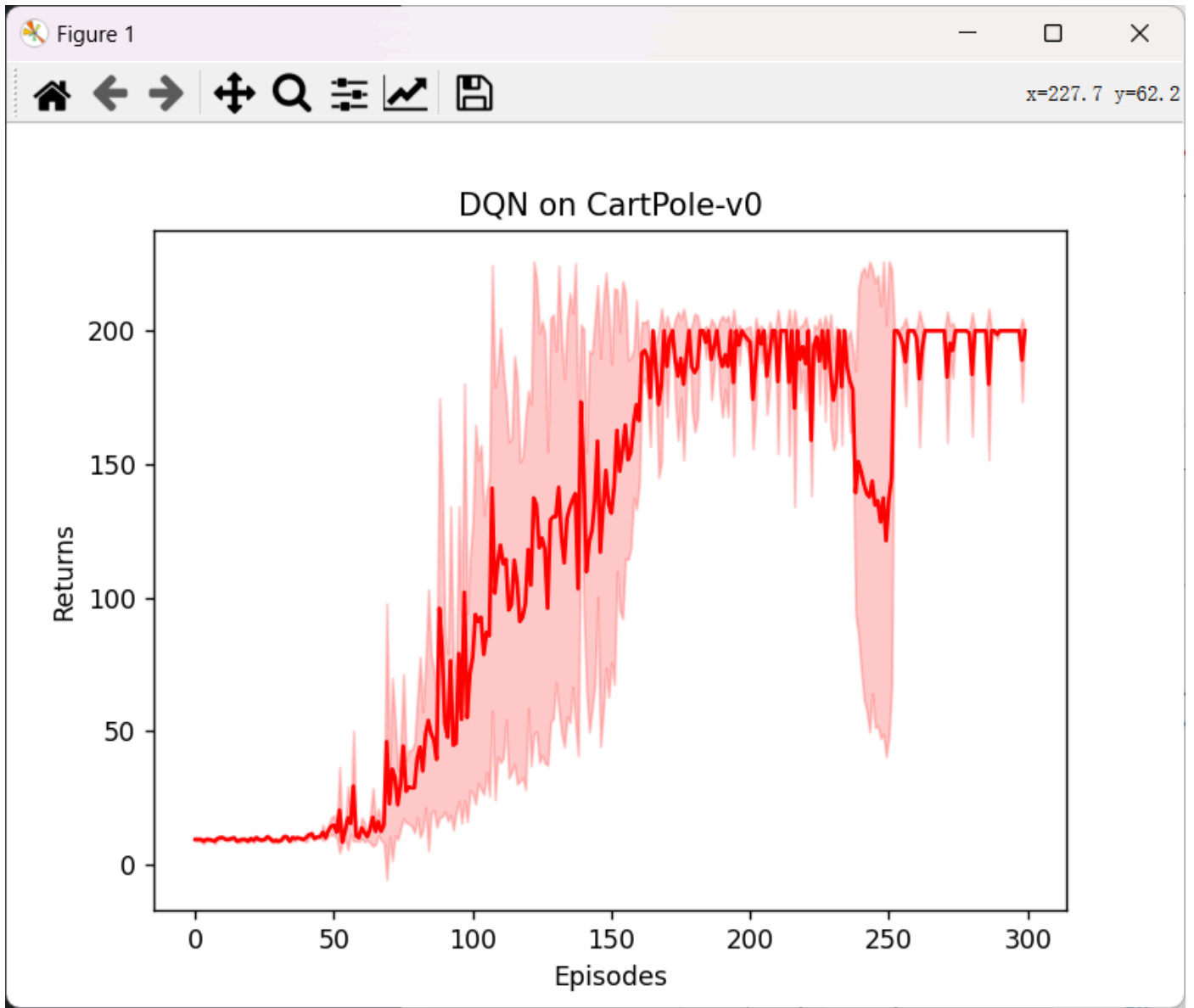
进行三次实验，求三次结果的平均值以及标准差阴影带如下：



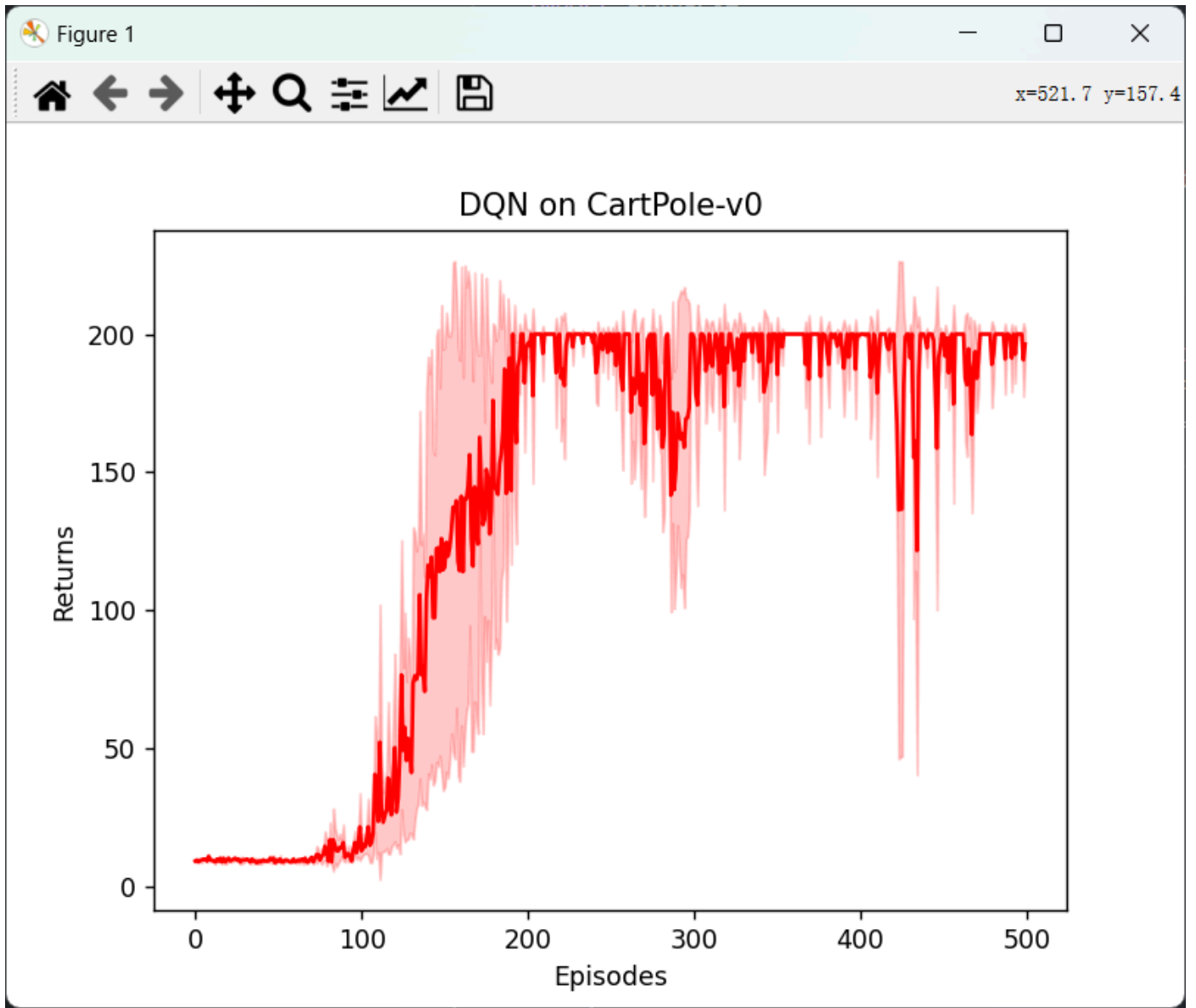
如果迭代次数设置为300次，则结果如下：



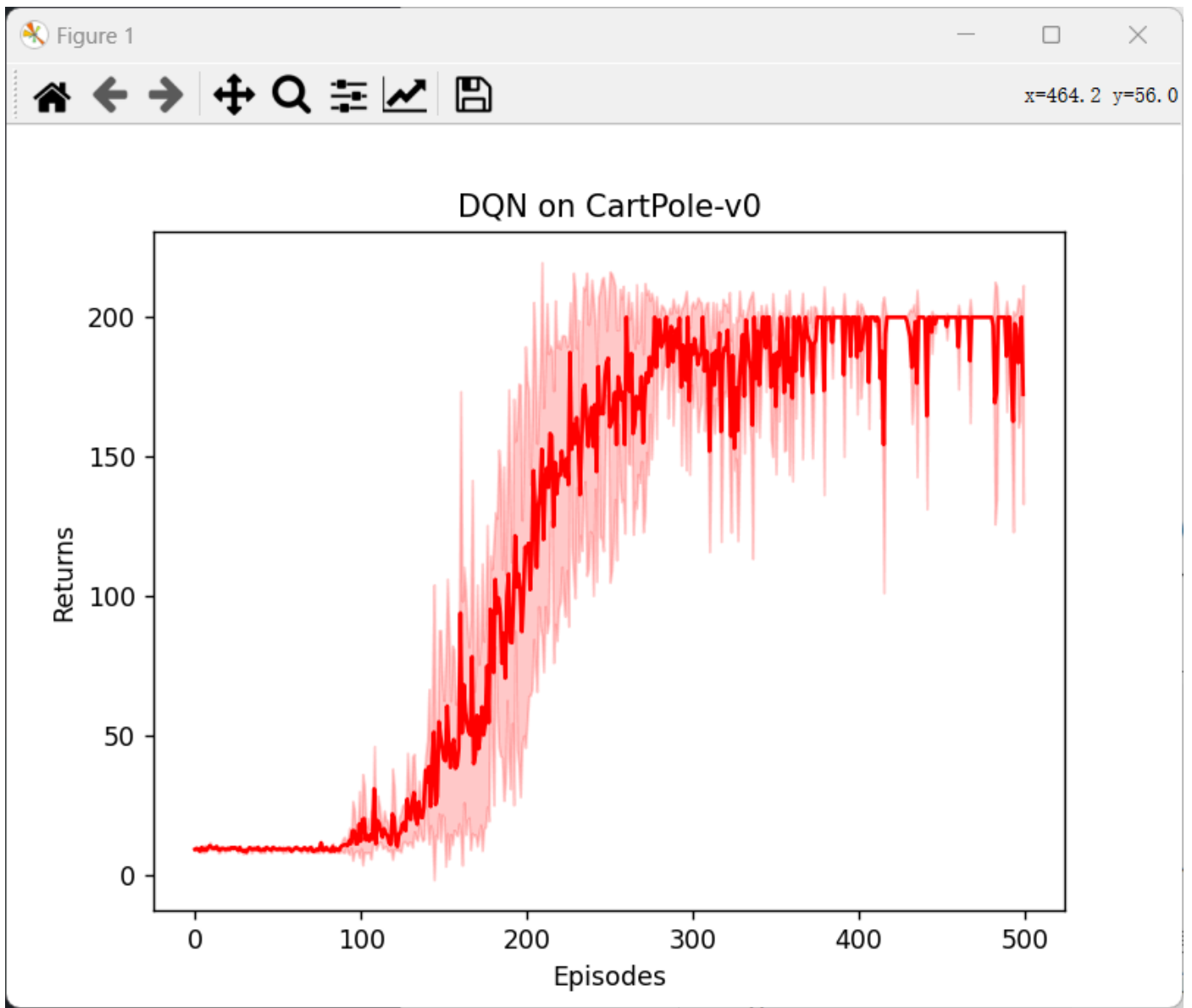
使用样本数量为32次，300次的实验结果如下，可以看见仍然是可以收敛与200。



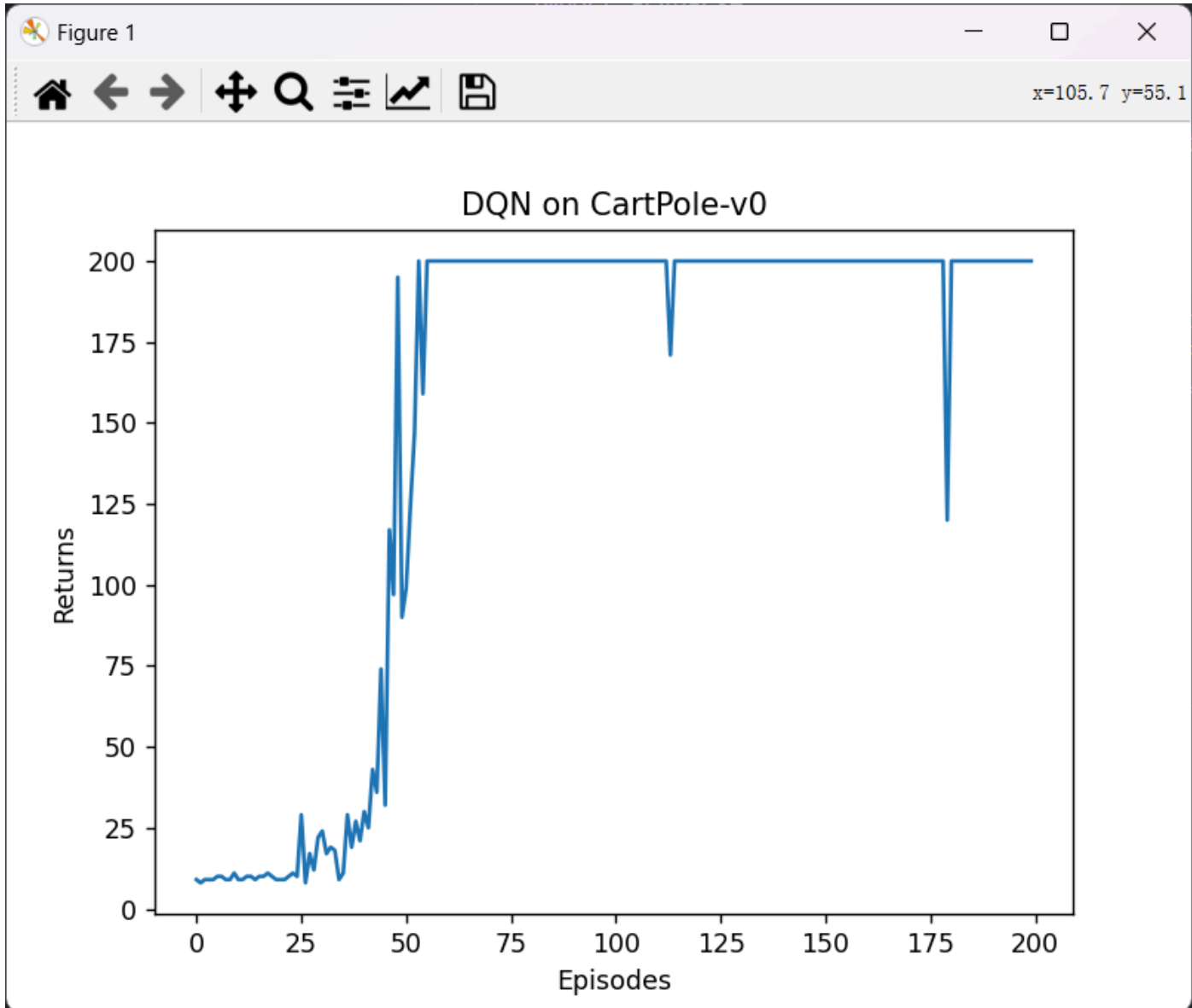
迭代500次，使用样本数量为32个时的结果图：

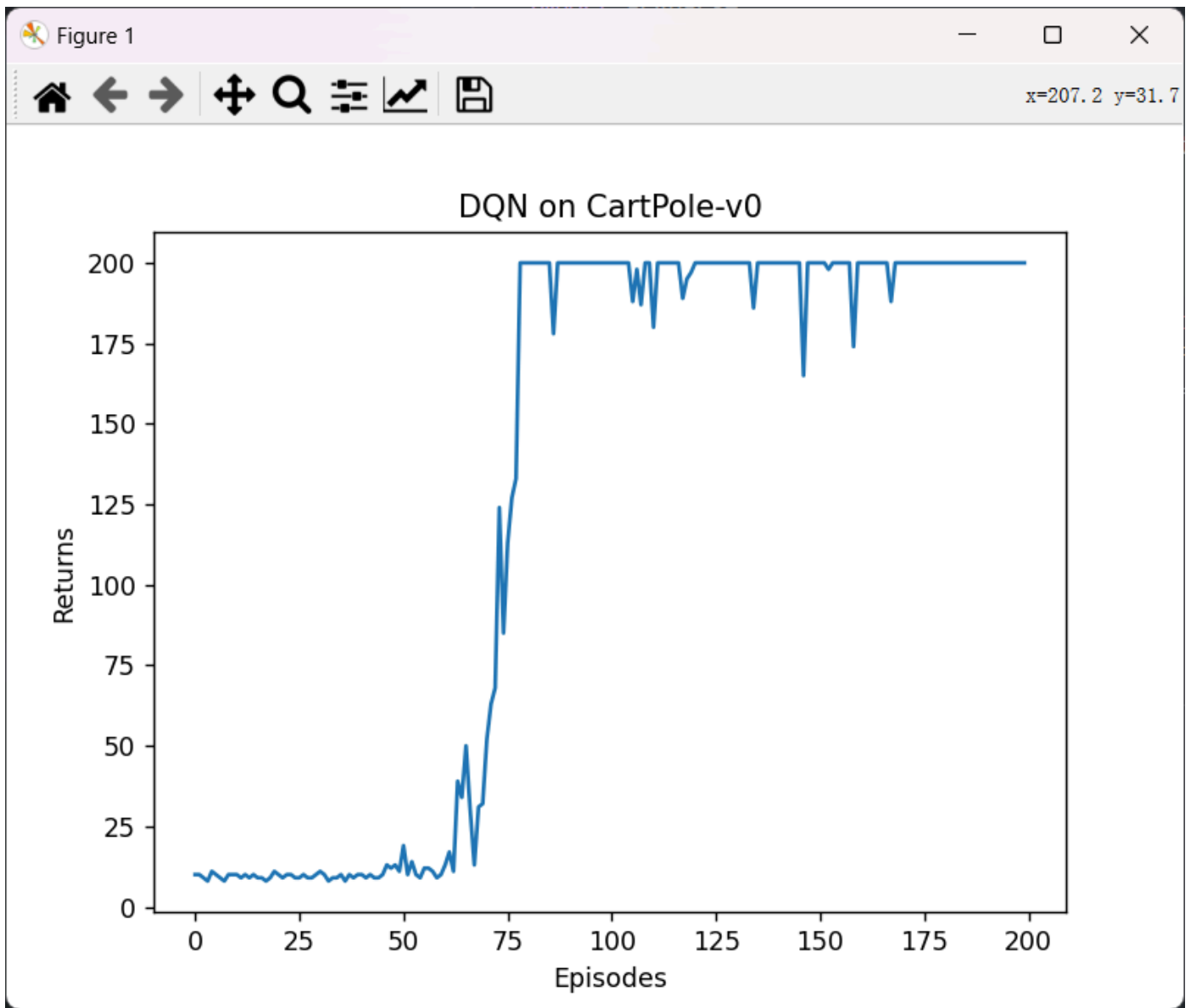


迭代500次，样本数量为16时的结果图：



附上单次测试时结果比较好的曲线图（学习率为0.03时的图片）：





2.评测指标展示及分析

主要从reward平均值与标准差曲线两个方面进行评测。

从reward平均值曲线可以看到，所用更新Q值的样本数目越多，收敛越快，`batch_size`为32时，大概在200次时开始收敛于200，当`batch_size`为16时，大概在300次左右收敛与200。虽然有波动但波动不会造成太大差距，总体是平稳收敛的，方差有波动但总体是稳定的。

四.参考资料:

https://blog.csdn.net/qq_37085158/article/details/125652566

<https://github.com/boyu-ai/Hands-on-RL/issues/72>

PS:本次实验我参考了https://blog.csdn.net/qq_37085158/article/details/125652566博客的基本思路和参数调整依据，但由于我的gym版本与其不匹配，主要导致env.reset()和env.step(action)返回值有差异，经由<https://github.com/boyu-ai/Hands-on-RL/issues/72>提供的方法成功进行了解决。如果想要复现的话，需要gym版本信息如下：

```
● PS C:\Users\86135\Desktop\DQN> pip show gym
Name: gym
Version: 0.26.2
Summary: Gym: A universal API for reinforcement learning environments
Home-page: https://www.gymnasium.dev/
Author: Gym Community
Author-email: jkterry@umd.edu
License: MIT
Location: C:\Users\86135\AppData\Roaming\Python\Python312\site-packages
Requires: cloudpickle, gym-notices, numpy
Required-by:
```

此外，不知什么原因，导致所给代码中导入Agent的代码总是报错，权衡之后，我将DQNAgent继承Agent进行了修改，使其不再继承Agent了。